



[AES HARDWARE CORE DESIGN]

Version: 1.1



FEBRUARY 11, 2026

**Presented By: Digital IC Design Team At Obour Engineering High Institute for
Engineering & Technology – Obour**

Academic Year: 2025 - 2026

1. Executive Summary:

This document presents the hardware design and implementation of an Advanced Encryption Standard (AES) cryptographic core optimized for **Industrial IoT (IIoT) Edge Gateways**. The primary design objective is to achieve a **low-power, resourceefficient architecture** while maintaining competitive performance suitable for real-time secure communication in constrained embedded systems.

The proposed architecture supports **AES-128**, **AES-192**, and **AES-256** through a parameterized and modular key expansion engine. Each AES transformation stage — SubBytes, ShiftRows, MixColumns, and Key Expansion — is implemented as an independent RTL module. This modular structure improves verification, reuse, and power-aware optimization while preserving predictable timing behavior.

To address the stringent performance and reliability requirements typical of IIoT deployments, the design emphasizes robust key scheduling, minimized latency, and hardware-efficient S-Box implementations. Two interchangeable S-Box architectures are provided: a lookup-table version optimized for high throughput and a Galois Field arithmetic version optimized for area and power trade-offs. This flexibility enables architectural exploration depending on industrial deployment requirements, ensuring continuous real-time secure communication under demanding operational conditions.

The resulting AES core achieves a balanced trade-off between throughput, silicon area, and power consumption. The design is fully synthesizable, adheres to RTL best practices, and is suitable for FPGA or ASIC integration in secure IIoT platforms. It also serves as an academic reference model for studying low-power cryptographic hardware design

2. Introduction:

Modern IIoT systems demand high-performance and secure encryption engines capable of sustaining continuous operation under strict reliability and throughput requirements. The Advanced Encryption Standard (AES) remains the global benchmark for symmetric cryptography, but traditional software implementations cannot meet the real-time demands of industrial embedded systems. Hardware-based AES cores are therefore essential to ensure low-latency, scalable, and resilient security for mission-critical industrial environments.

This project focuses on designing a hardware-optimized AES architecture that balances security, throughput, and power efficiency. By leveraging FPGA spatial parallelism and optimized arithmetic structures, the proposed design targets low-power **Industrial IoT (IIoT) Edge Gateways** while maintaining competitive encryption speed.

3. AES Algorithm Overview:

The Advanced Encryption Standard (AES) is a symmetric block cipher standardized by NIST and widely used in secure communication systems. AES operates on fixed-size data blocks of 128 bits and supports three key sizes: 128, 192, and 256 bits. The number of encryption rounds depends on the key size:

AES-128 → 10 rounds

AES-192 → 12 rounds

AES-256 → 14 rounds

Each encryption round consists of a sequence of transformations applied to a 4×4 byte matrix called the state. These transformations provide confusion and diffusion, ensuring strong cryptographic security.

The AES round structure includes:

3.1 SubBytes

A non-linear byte substitution step where each byte is replaced using an S-box.

The S-box is derived from multiplicative inversion in $\text{GF}(2^8)$ followed by an affine transformation. This step introduces non-linearity and protects against linear and differential attacks.

3.2 ShiftRows

A cyclic shift operation applied to the rows of the state matrix.

Each row is shifted by a different offset to spread byte dependencies across columns, increasing diffusion.

3.3 MixColumns

A matrix multiplication over $\text{GF}(2^8)$ applied to each column of the state.

This operation mixes the bytes within a column, providing inter-byte diffusion and improving avalanche behavior.

3.4 AddRoundKey

A bitwise **XOR** between the state and a round key generated by the key expansion module.

This step integrates the secret key into the encryption process.

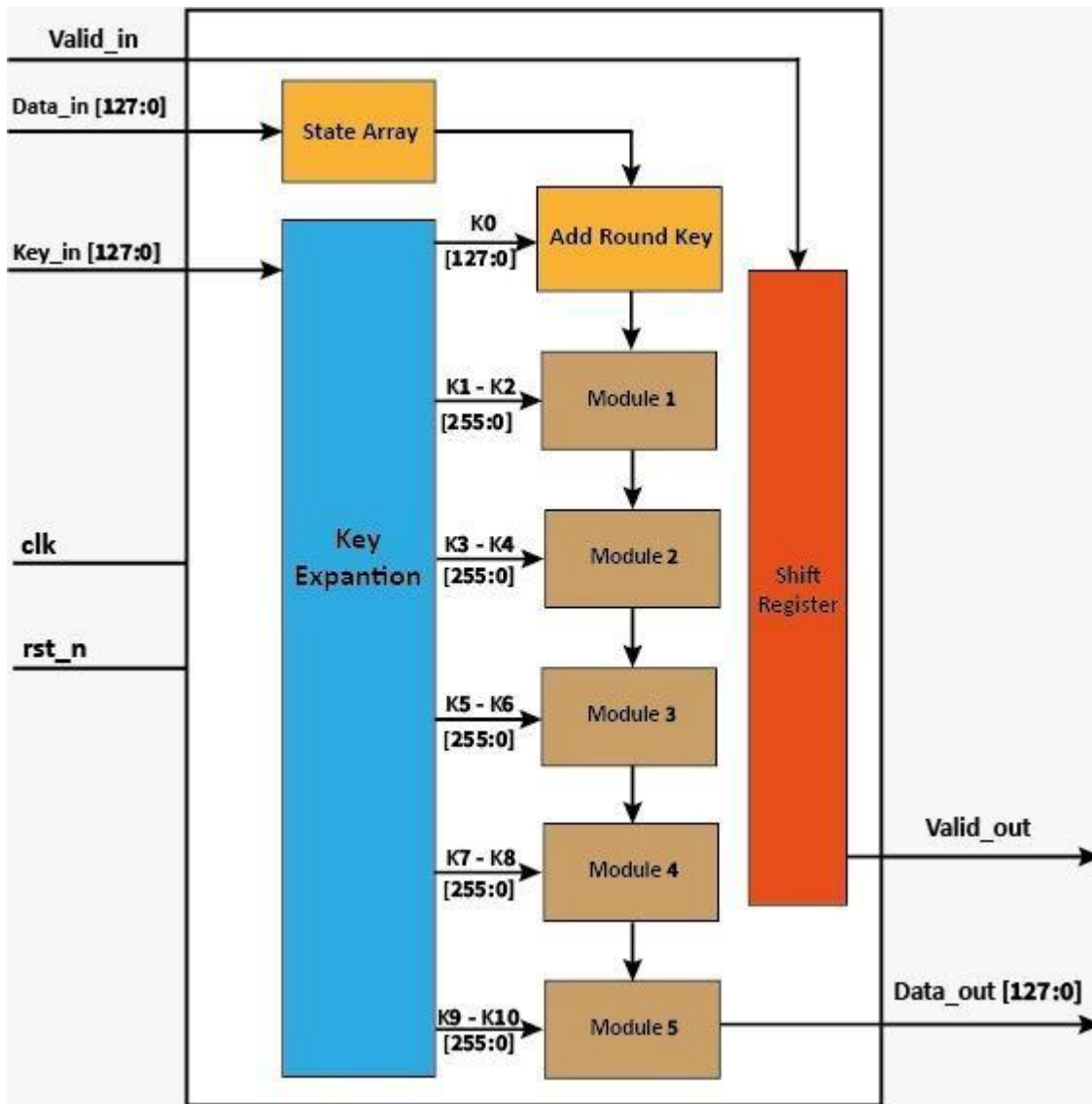


Figure 1 Block Diagram

4. Hardware Architecture Overview:

The AES hardware design is implemented as a modular architecture optimized for lowpower **Industrial IoT (IIoT) Edge Gateways** while maintaining competitive performance. The architecture separates the AES transformations into independent hardware blocks, enabling structured design, verification, and scalability.

The design follows a round-based architecture with precomputed round keys. Each transformation stage is implemented as a dedicated hardware module, allowing parallel development and easier verification.

The main hardware blocks are:

- Key Expansion Unit
- SubBytes Unit
- ShiftRows Unit
- MixColumns Unit
- Round Controller

These modules operate together to perform AES encryption in a deterministic pipelinelike structure.

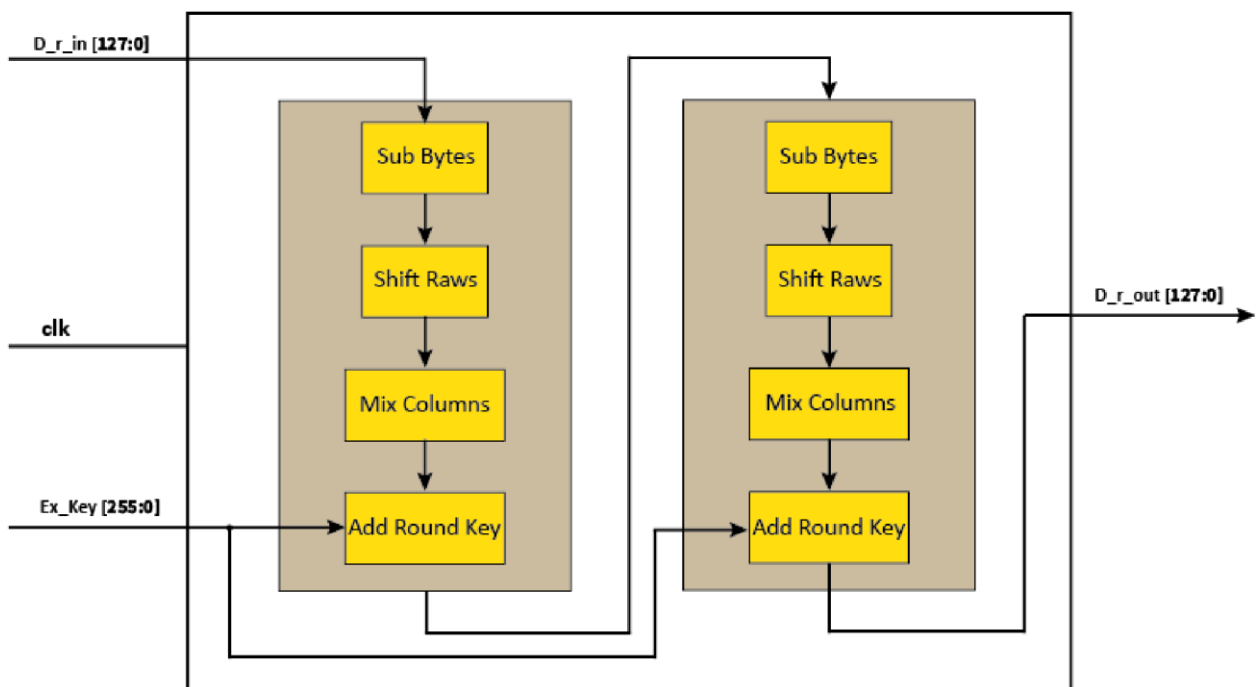


Figure 2 Module 1 to 4 (Round A & Round B) Structure

4.1 Key Expansion Architecture

The Key Expansion module precomputes round keys before encryption begins. This approach reduces runtime computation and improves throughput.

The design supports AES-128, AES-192, and AES-256 using a parameterized architecture.

The key schedule is implemented using:

RotWord transformation

SubWord using configurable S-box

Rcon round constants

XOR feedback chain

Two round keys are generated per cycle in the sequential architecture to accelerate key scheduling while maintaining hardware efficiency. Precomputing keys is especially beneficial in IIoT systems where deterministic timing, high throughput, and continuous reliability are critical. This approach ensures that industrial gateways can sustain real-time secure communication without performance bottlenecks, even under demanding operational workloads.

4.2 SubBytes Hardware Block

The SubBytes module implements the AES S-box using two selectable architectures:

GF arithmetic implementation (logic-based)

Lookup ROM implementation

The GF-based design minimizes memory usage and provides a balanced trade-off for industrial deployments, while the ROM-based version offers high-throughput substitution optimized for continuous real-time operation. This dual-architecture flexibility ensures that IIoT gateways can adapt to varying industrial workloads, balancing resource efficiency with speed and reliability under demanding

conditions. A design parameter allows switching between implementations without modifying the top-level architecture.

This modularity supports design exploration between:

Power optimization

Area optimization

Speed optimization

4.3 ShiftRows Hardware Block

ShiftRows is implemented as a pure wiring permutation without arithmetic logic. The operation is achieved using static byte routing, resulting in zero computational latency and minimal hardware cost.

This transformation introduces diffusion with negligible power overhead.

4.4 MixColumns Hardware Block

MixColumns performs matrix multiplication in **GF(2⁸)** using optimized **XOR** and **shift** networks. The implementation avoids lookup tables and uses combinational logic to reduce memory footprint.

The design prioritizes:

Low switching activity

Minimal gate count

Efficient critical path timing

This makes it suitable for embedded and IIoT cryptographic engines.

4.5 Control and Data Flow

The AES round controller orchestrates data movement between blocks. Each round executes in a fixed sequence:

SubBytes → ShiftRows → MixColumns → AddRoundKey

The final round omits MixColumns as required by the AES standard.

The architecture guarantees deterministic latency and predictable power consumption — key requirements for IIoT security systems.

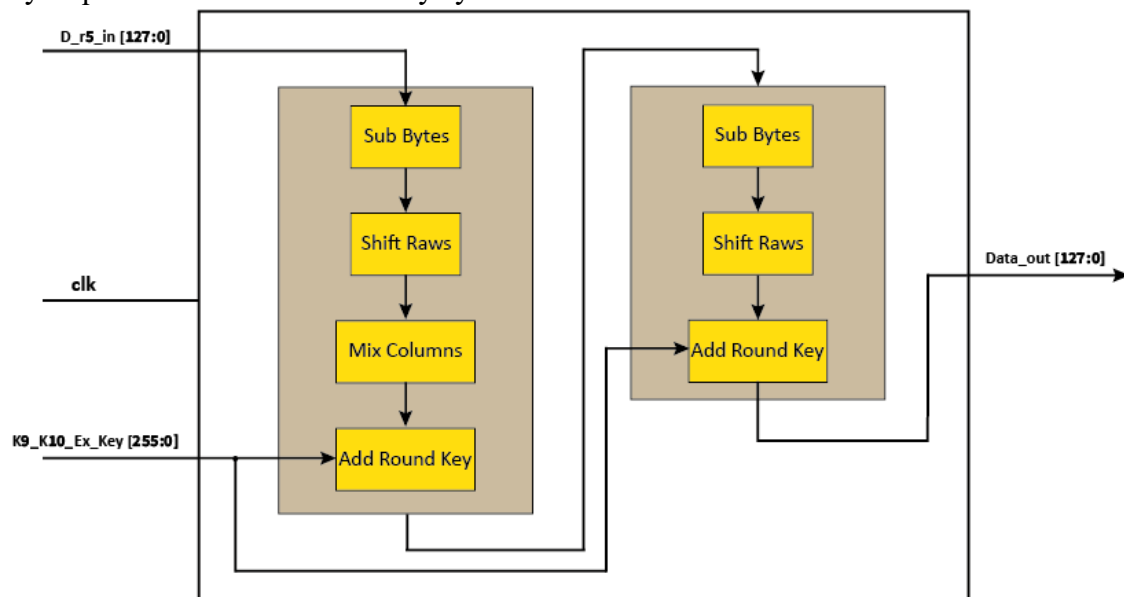


Figure3 Module 5 (Round A & Round B) Structure

5. AES Building Blocks Implementation:

5.1 S-Box (GF Implementation)

○ Design Overview

The AES Substitution Box (S-Box) is a nonlinear transformation used in every encryption round.

Instead of implementing the S-Box as a lookup table (ROM-based design), this work adopts a **GF(2⁸) arithmetic logic implementation**.

The goal of this approach is to:

- Reduce memory usage
- Minimize power consumption
- Improve hardware efficiency
- Enable IIoT-friendly lightweight encryption
- Maintain constant-time execution

The S-Box is implemented as a purely combinational logic circuit derived from the mathematical definition of AES: multiplicative inverse in **GF(2⁸)** followed by an affine transformation.

This design avoids table storage and replaces it with optimized **XOR/AND** logic networks.

○ Mathematical Background

The AES S-box transformation consists of two main operations:

1. Multiplicative inversion in the finite field **GF(2⁸)**
2. Affine transformation over **GF(2)**

Every input byte is treated as a polynomial over **GF(2)**:

$$a(x) = a_7x^7 + a_6x^6 + \dots + a_1x + a_0$$

The inversion is computed modulo the AES irreducible polynomial:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

The resulting byte is then passed through a linear affine mapping.

This hardware implementation expresses these operations using optimized Boolean equations.

○ Hardware Architecture

The S-box circuit is divided into three logical stages:

Stage 1 — Input Remapping

The input byte is internally reordered to match the optimized boolean network structure.

Stage 2 — GF Inversion Network

This stage implements the nonlinear core using **XOR** and **AND** gates. Intermediate **signals (y, t, z)** represent layered logic dependencies that compute the finite field inverse.

The circuit is based on a minimized logic representation to reduce gate count and switching activity.

Stage 3 — Affine Output Transformation

The final stage reconstructs the output byte using linear combinations of intermediate signals.

The result is the standard AES S-box output.

○ Timing Characteristics

The S-box is fully combinational:

- No clock dependency
- No memory elements
- Zero pipeline stages
- Deterministic propagation delay

This makes it suitable for high-speed and low-power cryptographic cores. ○

Verification Methodology

The design is verified using an exhaustive testbench.

All 256 possible input byte values are applied and compared against the official AES S-box reference table.

○ Design Advantages

- ROM-free implementation
- Low power consumption
- IIoT-optimized architecture
- Constant execution time
- Reduced hardware footprint
- Secure against timing attacks
- Fully synthesizable

S-Box GF Implementation

The module **sbox_gf** implements the AES Substitution Box (S-Box) using pure combinational logic based on finite field arithmetic over **GF(2⁸)**, instead of a lookup table.

This implementation replaces the traditional 256-entry ROM with optimized Boolean equations derived from the mathematical multiplicative inverse in GF(2⁸), followed by an affine transformation. The result is a memory-free S-Box tailored for high-reliability industrial cryptographic accelerators, enabling continuous real-time operation with reduced hardware overhead. Such an approach ensures scalability and resilience in IIoT environments where throughput and deterministic performance are critical.

The design is fully combinational, has **zero latency**, and is optimized for synthesis in FPGA and ASIC environments.

Module Interface module

```
sbox_gf (    input wire
           [7:0] data_i,    output wire
           [7:0] data_o
        );
```

The module accepts:

data_i: an 8-bit input byte data_o: the substituted

8-bit output byte

Each input byte is transformed independently according to the AES S-Box specification. The module contains no clock and no internal state.

Internal Signals

wire [7:0] x; wire

[7:0] s; wire

[21:1] y; wire

[67:0] t; wire

[17:0] z;

The internal wires form stages of the Boolean network:

x is the internal representation of the input byte

y contains **XOR** preprocessing signals t holds

nonlinear intermediate results z is an

AND reduction stage s stores the final substituted

byte

Together these signals implement the AES **S-Box** logic without memory.

Step 1: Input Mapping

```
assign  x  = {  
data_i[0], data_i[1], data_i[2],  
data_i[3], data_i[4],  
data_i[5], data_i[6], data_i[7]  
};
```

This section maps the input byte into internal bit ordering used by the optimized Boolean equations.

Functionally this is equivalent to: x

$= \text{data_i}$ but expressed explicitly to match the mathematical derivation of the SBox circuit.

Step 2: Intermediate Boolean Network

Example assignments:

```
assign y[14] = x[3] ^ x[5]; assign y[13]  
= x[0] ^ x[6]; assign t[2] = y[12] &  
y[15]; assign t[4] = t[3]  
^ t[2];
```

These lines implement the **nonlinear GF(2⁸)** inversion stage using Boolean logic.

Operations used:

XOR → linear combination in **GF(2)** **AND**

→ nonlinear multiplication chained

XOR/AND → finite field arithmetic

This stage replaces the ROM lookup by directly computing the multiplicative inverse mathematically.

The equations are minimized for hardware efficiency.

Logic Structure

The S-Box circuit can be divided into three logical layers:

XOR preprocessing stage (y signals)

Generates linear combinations of input bits to reduce later complexity.

Nonlinear inversion core (t signals)

Implements the multiplicative inverse in $\text{GF}(2^8)$, which is the cryptographic core of AES.

Reduction stage (z signals)

Combines intermediate terms to prepare for the affine transformation.

Step 3: Affine Transformation

assign $s[0] = t[59] \wedge t[63]$; assign $s[1]$

$= \sim t[64] \wedge s[3]$; assign $s[2] =$

$\sim t[55] \wedge t[67]$;

...

This stage performs the AES affine transform:

$S(\mathbf{x}) = \mathbf{A} \cdot \mathbf{x}^{-1} + \mathbf{b}$ Where:

A is a fixed matrix b is a

constant vector operations

are in $\text{GF}(2)$

This ensures diffusion and nonlinearity required for AES security.

Step 4: Output Reordering

assign $\text{data_o}[7] = s[0]$; assign

$\text{data_o}[6] = s[1]$;

...

```
assign data_o[0] = s[7];
```

This maps internal bit order back to AES standard byte format.

The final output matches the official AES S-Box lookup table exactly.

Functional Summary

The module performs:

Input byte

→ $\text{GF}(2^8)$ multiplicative inverse

→ affine transformation

→ output byte

All implemented using combinational logic with no memory.

Properties of the design: zero

clock cycles

no ROM low power

hardware optimized

FPGA/ASIC friendly IoT

suitable constant latency

deterministic behavior

Design Motivation

Compared to a ROM-based S-Box:

ROM implementation requires memory storage and higher power consumption.

GF logic implementation replaces memory with Boolean gates, reducing power and area while maintaining cryptographic correctness.

This approach is preferred in secure embedded hardware where memory is limited and power efficiency is critical.

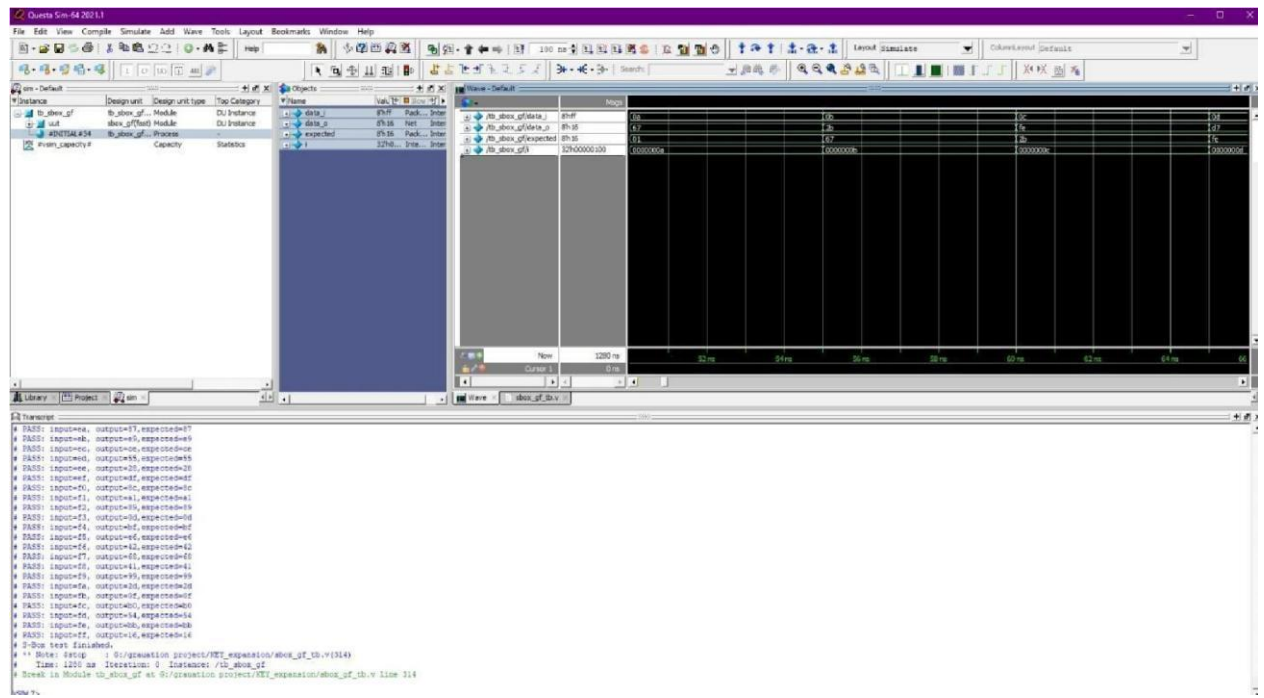


Figure 4 Test Bench S-BOX_GF

5.2 S-Box ROM Implementation (Lookup Table (

The module `sbox_assign` implements the AES Substitution Box using a ROMstyle lookup table. Unlike the GF arithmetic implementation, this design directly stores the official AES S-Box values as constants and performs substitution by indexing memory.

This approach is conceptually simpler than the GF implementation and matches the mathematical S-Box table defined in the AES standard. It is equivalent to reading from a 256-byte ROM.

The module processes a 32-bit word in parallel by applying four S-Boxes simultaneously, one for each byte.

```

Module      Interface      module
sbox_assign(  input wire [31:0]
sboxw,       output wire [31:0]
new_sboxw

);

```

The module accepts:

sboxw: a 32-bit input word (4 bytes) new_sboxw:

a 32-bit output word after substitution

Each byte is substituted independently.

This design is fully combinational and has no clock or internal state.

S-Box Memory Declaration wire [7:0] sbox [0:255];

This line declares a 256-entry array of 8-bit wires.

Functionally, this behaves like a ROM:

address → substituted byte

Each index corresponds to an AES S-Box

value. Example: sbox[0x00] = 0x63

sbox[0x01]

= 0x7C

...

This matches the official AES S-Box table exactly.

Parallel Byte Substitution assign

new_sboxw[31:24] = sbox[sboxw[31:24]]; assign

new_sboxw[23:16] = sbox[sboxw[23:16]]; assign

new_sboxw[15:8] = sbox[sboxw[15:8]]; assign

new_sboxw[7:0] = sbox[sboxw[7:0]];

Each byte of the 32-bit input is used as an index into the S-Box table.

This performs four substitutions in parallel:

byte0 → S-Box → output0 byte1 →

S-Box → output1 byte2 → S-

Box → output2 byte3 → S-Box

→ output3

This matches the AES SubBytes step for a 32-bit word.

Parallel execution improves throughput and reduces latency.

S-Box Initialization Example

entries:

```
assign sbox[8'h00] = 8'h63; assign sbox[8'h01]  
= 8'h7c; assign sbox[8'h02]  
= 8'h77;
```

...

```
assign sbox[8'hff] = 8'h16;
```

These assignments define the entire AES S-Box lookup table.

Each entry is constant and corresponds to the AES standard.

Mathematically:

$$S(x) = \text{affine}(\text{inverse}(x))$$

But here the inverse + affine transform is precomputed and stored.

This trades logic complexity for memory usage.

Functional Behavior

The module performs:

32-bit input word

→ split into 4 bytes

→ each byte indexed into ROM → substituted

32-bit output word

No arithmetic is performed at runtime.

The S-Box transformation is retrieved directly from constant storage.

Architectural Properties

This design has the following characteristics:

zero clock latency purely
combinational memorybased
substitution constant timing simple
implementation easy verification
FPGA ROM-friendly
predictable synthesis

Compared to the GF logic S-Box:

○ ROM S-Box:

simpler design higher memory
usage fewer logic
gates

○ GF S-Box: no memory more logic lower power in embedded systems

Both produce identical cryptographic output.

Design Motivation

This implementation is useful when:

area is not critical

ROM is cheap design simplicity is preferred

fast verification is needed synthesis tools

optimize ROM efficiently

It is ideal for educational designs and FPGA prototypes.

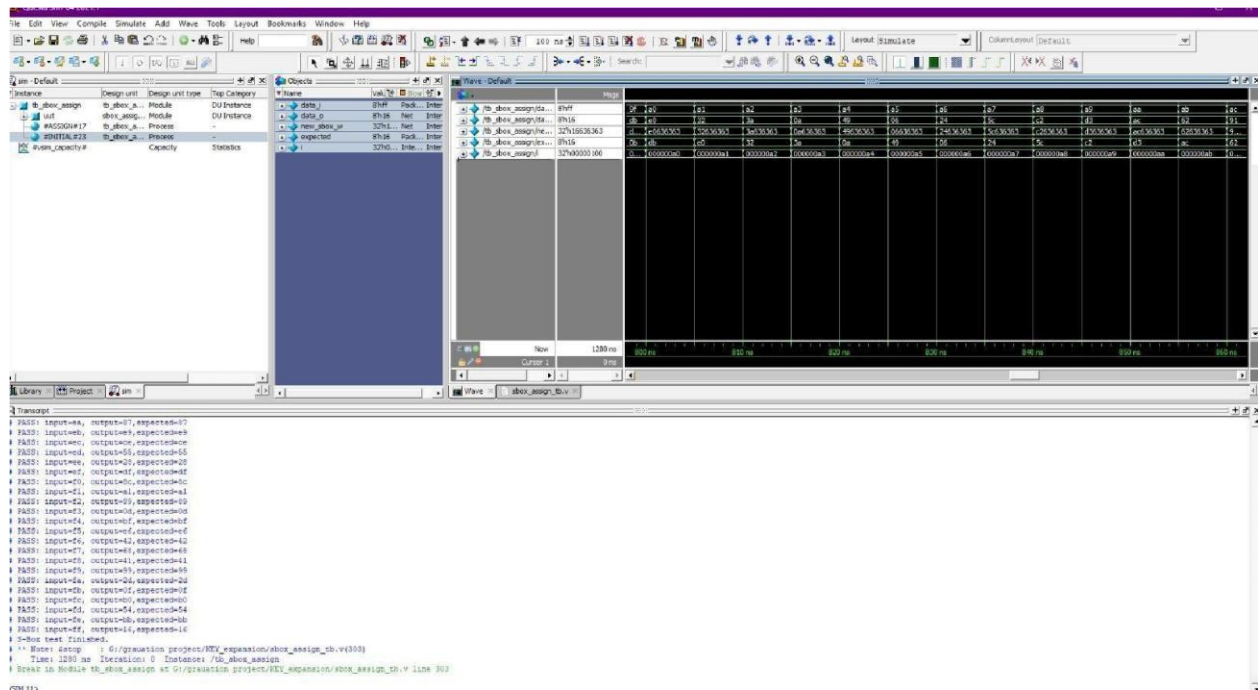


Figure 5 Test Bench S-BOX_Assign

5.3 g_function Module (Key Expansion Core Primitive)

The g_function module implements the core nonlinear transformation used in the AES Key Expansion algorithm. It performs two operations:

RotWord → byte rotation

SubWord → S-Box substitution

This module is applied repeatedly during AES key schedule generation and is responsible for introducing nonlinearity and diffusion into the round keys.

The design supports two interchangeable S-Box architectures:

GF arithmetic implementation (logic-based)

ROM lookup implementation

The choice is controlled by a parameter.

Module Interface module g_function #(parameter

use_gf = 1) (input wire [31:0] word_in,
output wire [31:0] word_out);

Inputs:

word_in → 32-bit word from key schedule

Output:

word_out → transformed word after g_function

Parameter:

use_gf = 1 → uses GF arithmetic S-Box use_gf

= 0 → uses ROM lookup S-Box

This parameter allows architectural exploration without modifying higher-level logic.

Step 1: RotWord Operation

wire [31:0] shiftword_out; assign

shiftword_out = {

word_in[23:16],

word_in[15:8], word_in[7:0],

word_in[31:24] };

This performs a left byte rotation:

[b0 b1 b2 b3] → [b1 b2 b3 b0]

Purpose:

spreads byte dependencies improves

diffusion matches AES standard key schedule

This is a pure wiring operation with zero logic delay.

Step 2: SubWord Operation

wire [31:0] sword_out;

This signal stores the substituted result after applying the S-Box to all bytes.

Conditional S-Box Architecture

generate if (use_gf == 1) begin

This compile-time condition selects the implementation. It does not generate runtime branching. The choice is fixed during synthesis. GF Arithmetic Implementation

```

genvar i; for (i = 0; i < 4; i = i +
1) begin : sbbox_loop    sbbox_gf sbbox_ins (

```

```

        .data_i(shiftword_out[8*i+: 8]),
        .data_o(sword_out[8*i+: 8])
    );
end

```

end

This loop instantiates four independent S-Boxes.

Each byte is processed in parallel: byte0

```

→ S-Box → output0 byte1
→ S-Box → output1 byte2 → S-
Box → output2 byte3 → S-Box
→ output3

```

The [8*i+: 8] syntax selects an 8-bit slice.

Example:

```

i=0 → bits [7:0] i=1 →
bits [15:8] i=2
→ bits [23:16] i=3
→ bits [31:24]

```

Parallel execution ensures minimal latency. ROM

Lookup Implementation

```

else begin    sbbox_assign

```

```

    sbbox_ins (
        .sboxw(shiftword_out),
        .new_sboxw(sword_out)
    );
end

```

end

Instead of four GF S-Boxes, a single 32-bit lookup module is used.

This implementation internally performs four byte substitutions using a ROM table.

This version is: simpler memorybased

easier to verify

FPGA friendly

Step 3: Final Output Assignment assign word_out

= sword_out;

The substituted word becomes the final output.

No extra processing is performed.

Functional Behavior

The full transformation is:

input word

→ rotate bytes

→ apply S-Box to each byte

→ output transformed word

This exactly matches the AES Key Expansion $g()$ function.

Cryptographic Role

The g function:

introduces nonlinearity prevents linear

key relationships increases resistance

to cryptanalysis ensures avalanche

behavior

Without this transformation, AES key schedule would be insecure. Architectural

Characteristics

This module is:

- fully combinational zero clock
- latency parallel byte
- processing parameterizable
- architecture synthesis friendly
- reusable across AES variants

5.4 Sequential Key Expansion Module

The keyExpansion_seq2 module implements a sequential AES key expansion engine supporting:

AES-128

AES-192

AES-256

It generates expanded round keys iteratively using a clocked process and stores them internally before packing them into output blocks.

This design is optimized for:

low area low

power

IoT-friendly hardware sequential reuse

of logic

Instead of computing all words in parallel, the design expands keys one word per clock cycle.

Module Header module

```
keyExpansion_seq2 #  
  
(  
    parameter Nk = 4,  
    parameter Nb = 4,    parameter
```

$$Nr = (Nk==4) ? 10 : \\ (Nk==6) ? 12 : 14 \\)$$

Parameters:

$Nk \rightarrow$ number of 32-bit words in original key

4 $\rightarrow$ AES-128

6 $\rightarrow$ AES-192

8 $\rightarrow$ AES-256

$Nb \rightarrow$ block size in words (fixed = 4 for AES)

$Nr \rightarrow$ number of AES rounds

10 for AES-128

12 for AES-192

14 for AES-256

These parameters allow a single module to support all AES variants.

Module Ports input wire clk, input wire

[$Nk*32-$

1:0] KEY_in, output reg [$256*(Nr/2)-$

1:0] KEY_out clk $\rightarrow$ clock signal

KEY_in $\rightarrow$ original cipher key

128 / 192 / 256 bits depending on Nk

KEY_out $\rightarrow$ packed expanded round keys

keys stored in 256-bit pairs optimized for

pipeline feeding Derived Parameter localparam

TOTAL_W = $Nb*(Nr+1);$

TOTAL_W = total number of expanded 32-bit words required.

AES key schedule always generates:

$Nr + 1$ round keys

Each round key = 4 words

So:

TOTAL_W = 4 × (Nr + 1) Internal

Storage reg [31:0] W [0:TOTAL_W-1];

Array W stores all expanded key words.

Each element is one 32-bit word. This acts
as the key schedule memory.

reg [5:0] i = 0; i is the word
index counter.

It tracks which expanded word is currently being generated.

reg [31:0] temp;

Temporary register used during transformation.

Stores intermediate word before **XOR**.

reg done = 0;

Flag indicating that expansion is complete.

Prevents recomputing output repeatedly.

integer r;

Loop index used when packing round keys into output.

g_function Instance wire

[31:0] g_out; g_function gfun

```
(  
    .word_in( (i>0) ? W[i-1] : 32'h0 ),  
    .word_out(g_out)  
);
```

g_function performs:

RotWord + SubWord + Rcon preparation

It operates on the previous word W[i-1].

Special case:

When i = 0 → no previous word exists

So zero is fed to avoid invalid indexing.

Rcon Table localparam [31:0] Rcon

`[0:15] = '{ ... }';`

Rcon is the AES round constant table.

Used only when: `i mod Nk ==`

`0`

It introduces round-dependent nonlinearity.

Index 0 is unused (dummy entry).

Sequential Process always

`@(posedge clk)`

The key expansion runs synchronously.

Each clock cycle generates one word.

The process has three phases.

Phase 1: Load Original Key

`if (i < Nk) begin`

`W[i] <= KEY_in[32*(Nk-1-i) +: 32];`

`i <= i + 1; end`

The first Nk words come directly from the input key.

This loads the initial cipher key into W.

The indexing reverses word order to match AES format.

After Nk cycles:

Original key is stored in `W[0 ... Nk-1]`

Phase 2: Generate Expanded Words else if

`(i < TOTAL_W)`

This phase performs AES key expansion.

Base Temporary Word `temp = W[i-1];`

Start with previous word. This is the

default AES rule. `g_function` Condition

`if (i % Nk == 0) temp = g_out ^`

Rcon[(i/Nk)]; Every Nk words: apply

RotWord apply

SubWord

XOR with round constant

This is the core AES transformation.

AES-256 Special Case else if (Nk ==

8 && (i % Nk) == 4) temp =

g_out;

AES-256 requires an extra SubWord step.

No Rcon is applied here.

This matches the AES standard.

Word Generation Rule

$W[i] \leftarrow W[i-Nk] \oplus \text{temp};$

New word = word Nk positions earlier **XOR** transformed temp

This creates dependency across rounds.

$i \leq i + 1;$

Advance to next word.

Phase 3: Pack Output Keys

else if (!done)

Once expansion is finished:

Pack round keys into output. for

(r = 1; r < Nr; r = r + 2)

Each iteration packs two round keys.

Each round key = 128 bits

Two keys = 256 bits

$\text{KEY_out}[256*((Nr-r)/2) +: 256] \leftarrow \{$

$W[4*r], W[4*r+1], W[4*r+2], W[4*r+3],$

$W[4*(r+1)], W[4*(r+1)+1],$

$W[4*(r+1)+2], W[4*(r+1)+3]$

$\};$

This concatenates:

Round r key

Round r+1 key

Into one 256-bit block.

Keys are packed in reverse order for pipeline feeding. done

≤ 1 ;

Marks completion.

Prevents repeated packing.

Functional Summary The module:

Loads original key

Iteratively expands words

Applies AES g_function when required

Handles AES-256 special rule

Stores full key schedule

Packs output efficiently

All operations follow FIPS-197 AES standard.

Architectural Properties

This design is: sequential area

efficient low power clock

driven reusable across AES

variants IoT optimized

hardware friendly

It trades speed for energy efficiency, which matches IoT constraints.

Design Rationale

Sequential expansion:

✓ reduces combinational area

✓ reuses g_function hardware

✓ minimizes switching activity

✓ lowers power consumption

✓ simplifies routing

Ideal for embedded cryptographic engines.

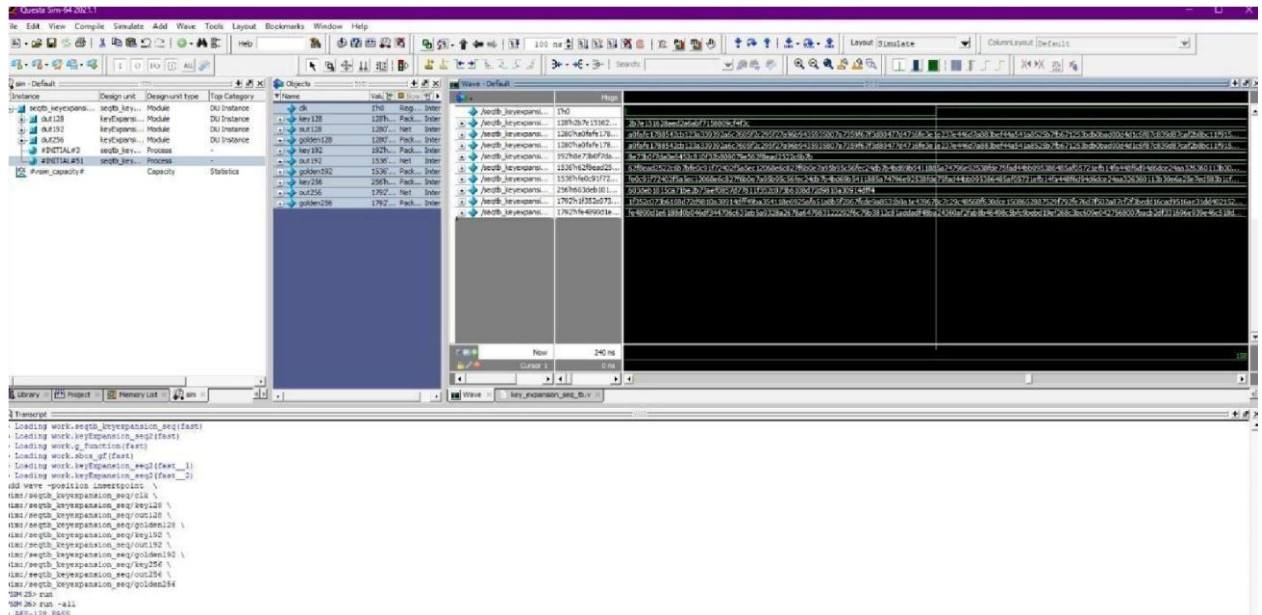


Figure 6 Test Bench Key Expansion

5.5 SubBytes Module

The sub_bytes module implements the AES SubBytes transformation, which is a nonlinear byte substitution stage.

This module:

processes a full 128-bit AES state substitutes each

byte using an S-box supports two

implementations:

GF arithmetic S-box (low memory, logic-based)

ROM lookup S-box (table-based)

The implementation is selected using a compile-time parameter.

Module Header module

sub_bytes(data_in,data_out); parameter

USE_gf=1;

USE_gf parameter selects the S-box implementation:

USE_gf = 1 → arithmetic GF S-box

USE_gf = 0 → lookup-table S-box

This allows design flexibility depending on area/speed tradeoff.

Ports input [127:0] data_in; output wire

[127:0] data_out ; data_in → 128-bit

AES state input data_out → substituted

128-bit output

AES state contains 16 bytes:

$16 \times 8 \text{ bits} = 128 \text{ bits}$

Each byte is substituted independently.

Generate Variable genvar i; genvar is a compile-time loop index.

Used only inside generate blocks.

It creates multiple hardware instances.

Generate Block generate

This block builds hardware at synthesis time.

Not runtime logic — pure structural replication.

Case 1: GF Arithmetic S-box if

(USE_gf==1)

Select arithmetic S-box implementation.

Byte-wise Replication for (i = 15; i >=

0; i = i - 1)

Loop generates 16 S-boxes.

Each iteration handles one byte.

Order goes from MSB → LSB.

This matches AES state layout.

Instance Creation `sbox_gf sbox_gf`

```
(  
    .data_i(data_in[8*i+: 8]),  
    .data_o(data_out[8*i+: 8]) );
```

For each byte:

- extract 8-bit slice from input pass into one
- S-box connect output back to same byte
- position

The operator:

`[8*i+: 8]`

means:

- start at bit $8*i$ take
- 8 bits forward
- This cleanly slices bytes.

Case 2: Lookup Table S-box

else

Select ROM-based S-box implementation. Word-wise Replication for $(i = 3; i \geq 0; i = i - 1)$
Loop generates 4 instances.

Each instance processes: 32

bits = 4 bytes

Total:

$4 \times 32 \text{ bits} = 128 \text{ bits Instance}$

Creation `sbox_assign sbox_assign`

```
(  
    .sboxw(data_in[32*i+:32]),
```

```
.new_sboxw(data_out[32*i+:32])
```

```
);
```

Each instance substitutes one 32-bit word.

Internally, that module contains 4 byte S-boxes.

This reduces instance count.

End Generate endgenerate

Closes compile-time hardware replication.

Functional Behavior

The module performs:

$\text{SubBytes}(\text{state}) = \text{S}(\text{state}[0]) \parallel \text{S}(\text{state}[1]) \parallel \dots \parallel \text{S}(\text{state}[15])$

Where:

$\text{S}(x)$ = AES S-box substitution All 16 bytes are

processed in parallel.

No sequential logic.

Pure combinational block.

Architectural Properties

The design is: fully

parallel

combinational no

clock required

one-cycle latency

scalable

configurable **This**

dualarchitectu

re design

allows:

optimization for IoT

power flexibility

for

FPGA vs ASIC

tradeoff between

logic vs memory

experimentation

in verification

Ideal for academic evaluation and design comparison.

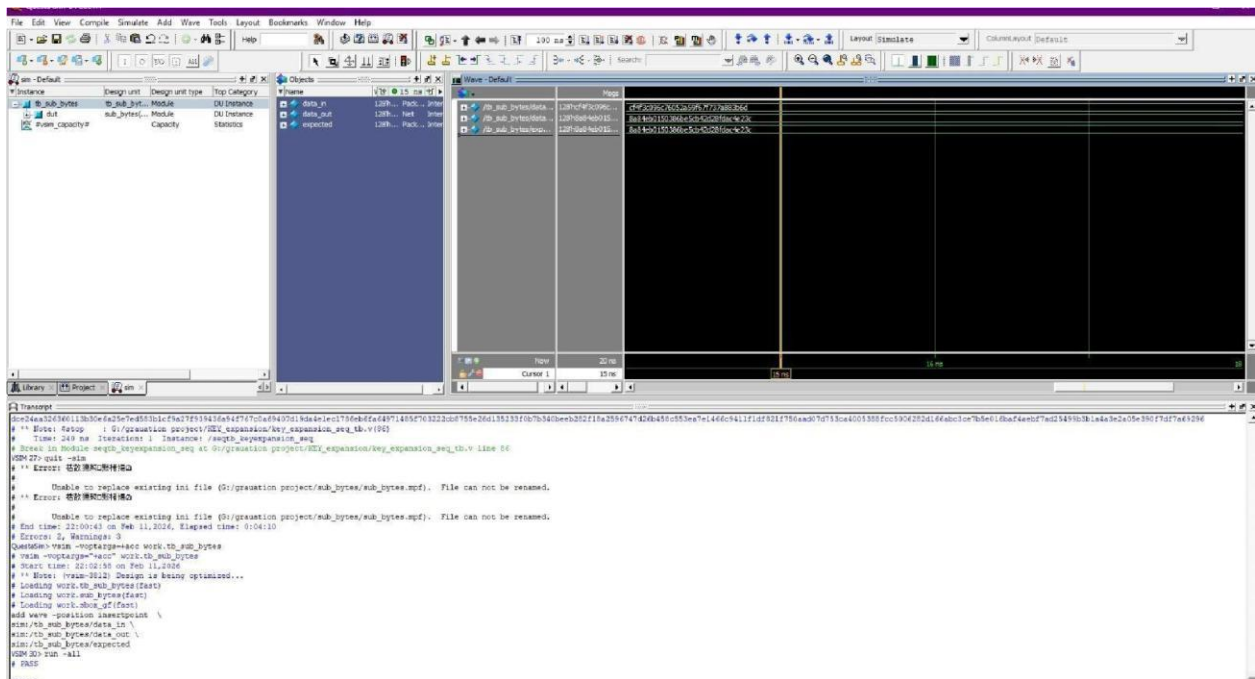


Figure 7 Test Bench SubBytes

5.6 ShiftRows Module

The shift_rows module implements the AES ShiftRows transformation.

ShiftRows is a byte permutation stage that:

rearranges the AES state

introduces inter-column diffusion

increases cryptographic mixing

prepares data for MixColumns

This implementation is:

- ✓ fully combinational
- ✓ zero latency
- ✓ direct wiring (no logic gates)
- ✓ optimized for speed
- ✓ optimized for low power

Module Header

```
module shift_rows (    input wire
[127:0] state_in,    output wire
[127:0] state_out
);
```

state_in → 128-bit AES state from SubBytes
state_out → permuted state to MixColumns

AES state is a 4×4 byte matrix:

Each element = 1 byte	S0 S4 S8 S12
Total = 16 bytes = 128 bits	S1 S5 S9 S13
ShiftRows Algorithm	S2 S6 S10 S14
	S3 S7 S11 S15

AES defines cyclic left shifts per row:

Row 0 → shift by 0

Row 1 → shift by 1

Row 2 → shift by 2

Row 3 → shift by 3

Mathematically:

$\text{state}[r][c] \rightarrow \text{state}[r][(c+r) \bmod 4]$

This is pure permutation.

No arithmetic involved.

Row 0 — No Shift assign $\text{state_out}[127:120] =$

$\text{state_in}[127:120]$; assign $\text{state_out}[95:88] =$

$\text{state_in}[95:88]$; assign $\text{state_out}[63:56] = \text{state_in}[63:56]$;

assign $\text{state_out}[31:24] = \text{state_in}[31:24]$;

Row 0 remains unchanged.

These bytes pass through directly.

This preserves the first row.

Row 1 — Shift Left by 1 assign $\text{state_out}[119:112] =$

$\text{state_in}[87:80]$; assign $\text{state_out}[87:80] =$

$\text{state_in}[55:48]$; assign $\text{state_out}[55:48] =$

$\text{state_in}[23:16]$; assign $\text{state_out}[23:16] =$

$\text{state_in}[119:112]$;

This rotates row 1:

$[a \ b \ c \ d] \rightarrow [b \ c \ d \ a]$

Each assignment performs byte rotation.

No logic — only wiring.

Row 2 — Shift Left by 2 assign $\text{state_out}[111:104] =$

$\text{state_in}[47:40]$; assign $\text{state_out}[79:72] =$

$\text{state_in}[15:8]$; assign $\text{state_out}[47:40] =$

$\text{state_in}[111:104]$; assign $\text{state_out}[15:8] =$

$\text{state_in}[79:72]$; This performs:

$[a\ b\ c\ d] \rightarrow [c\ d\ a\ b]$ Two-byte

cyclic shift.

Equivalent to swapping pairs.

Row 3 — Shift Left by 3 assign state_out[103:96]

= state_in[7:0]; assign state_out[71:64] =
state_in[103:96]; assign state_out[39:32] =
state_in[71:64]; assign state_out[7:0] =
state_in[39:32];

This is:

$[a\ b\ c\ d] \rightarrow [d\ a\ b\ c]$ Equivalent to
right shift by 1. Still a cyclic rotation.

Architectural Characteristics

This module is:

- ✓ purely combinational
- ✓ zero logic depth
- ✓ only wiring
- ✓ no LUT cost
- ✓ no clock
- ✓ no memory
- ✓ zero switching overhead

It is optimal for: IoT low-

power AES high-
speed datapaths

FPGA routing efficiency

ASIC area reduction

ShiftRows consumes almost no hardware.

Security Role in AES

ShiftRows ensures: inter-column

diffusion resistance to linear attacks

byte mixing before

MixColumns

Without ShiftRows:

AES becomes weak.

It is essential for avalanche behavior.

Design Rationale

This implementation avoids:

✗ multiplexers

✗ loops

✗ temporary registers

✗ extra logic

Each output bit is a direct wire.

This guarantees: minimum

latency minimum

power maximum

speed

Ideal for embedded AES cores.

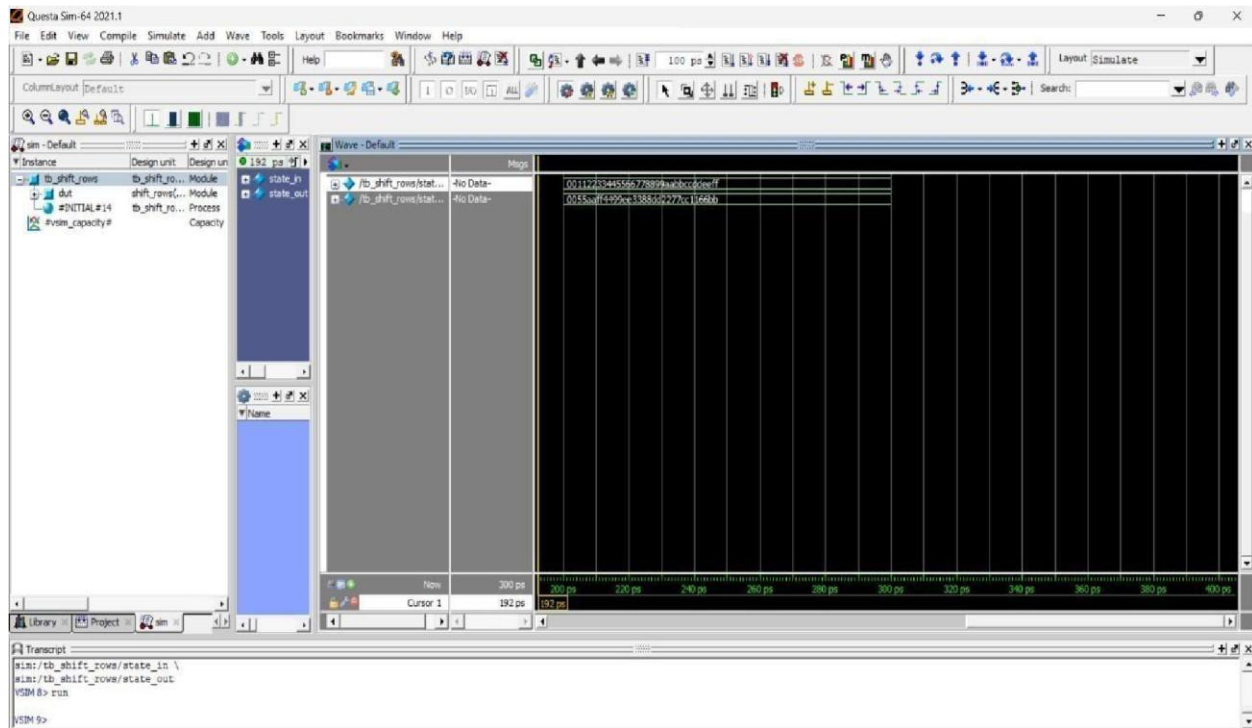


Figure 8 Test Bench ShiftRows

5.7 MixColumns Module

The MixColumns module implements the AES MixColumns transformation.

MixColumns is a linear transformation operating in:

$GF(2^8)$ finite field arithmetic

It provides:

- ✓ strong diffusion
- ✓ byte mixing inside each column
- ✓ resistance to statistical attacks
- ✓ avalanche effect

Each AES column is multiplied by a fixed matrix:

	02	03	01	01	
	01	02	03	01	
	01	01	02	03	

This module performs that multiplication in hardware.

```
Module      Header                module
    MixColumns(stateIn, stateOut);  input [127:0]
    stateIn;  output [127:0]
    stateOut;

    stateIn → 128-bit AES state stateOut →
    mixed AES state AES state is processed
    column-by-column.

    Each column = 4 bytes = 32 bits
    There are 4 columns total.

GF(28) Multiplication — xtime Function function
    [7:0] xtime;

    input [7:0] in;      if(in[7] == 1)
xtime = (in << 1) ^ 8'h1B;  else xtime = in
    << 1; endfunction

    This function multiplies a byte by 2 in GF(28).
```

Mathematical rule:

$$x \cdot 2 \bmod (x^8 + x^4 + x^3 + x + 1)$$

If MSB is 1 → overflow occurs → reduce using AES polynomial 0x1B.

This is standard AES finite field reduction.

$$\text{xtime}(\text{in}) = 2 \cdot \text{in}$$

Multiplication by 3 is implemented as: $3 \cdot x$

$= 2 \cdot x \text{ XOR } x$ Which is why you see:

xtime(x)

$\wedge x$

Column Processing Loop genvar i; generate for(i = 0; i < 4; i = i + 1) begin: mixColumnsLoop This loop processes the 4 AES columns in parallel.

Each iteration handles one column.

Fully combinational.

No clock.

No latency.

First Row Equation assign stateOut[32*i+24+:8] = xtime(stateIn[32*i+24+:8])

$\wedge$

(xtime(stateIn[32*i+16+:8]) $\wedge$ stateIn[32*i+16+:8]) $\wedge$
stateIn[32*i+8+:8] $\wedge$ stateIn[32*i+:8];

Implements:

$2 \cdot a0 + 3 \cdot a1 + 1 \cdot a2 + 1 \cdot a3$

Where:

a0 = row 0 byte a1

= row 1 byte a2 =

row 2 byte a3 = row

3 byte

This matches AES matrix multiplication.

Second Row Equation

assign stateOut[32*i+16+:8] = xtime(stateIn[32*i+16+:8])

$\wedge$

(xtime(stateIn[32*i+8+:8]) $\wedge$ stateIn[32*i+8+:8]) $\wedge$
stateIn[32*i+:8] $\wedge$ stateIn[32*i+24+:8];

Implements:

$$1 \cdot a_0 + 2 \cdot a_1 + 3 \cdot a_2 + 1 \cdot a_3$$

Rotation of coefficients.

Third Row Equation

$$\text{assign stateOut}[32*i+8+:8] = \text{xtime}(\text{stateIn}[32*i+8+:8])$$

^

$$(\text{xtime}(\text{stateIn}[32*i+:8]) \wedge \text{stateIn}[32*i+:8]) \wedge \text{stateIn}[32*i+24+:8]$$

$$\wedge \text{stateIn}[32*i+16+:8];$$

Implements:

$$1 \cdot a_0 + 1 \cdot a_1 + 2 \cdot a_2 + 3 \cdot a_3$$

Fourth Row Equation

$$\text{assign stateOut}[32*i+:8] = \text{xtime}(\text{stateIn}[32*i+:8])$$

^

$$(\text{xtime}(\text{stateIn}[32*i+24+:8]) \wedge \text{stateIn}[32*i+24+:8]) \wedge \text{stateIn}[32*i+16+:8]$$

$$\wedge \text{stateIn}[32*i+8+:8];$$

Implements:

$$3 \cdot a_0 + 1 \cdot a_1 + 1 \cdot a_2 + 2 \cdot a_3$$

Completes matrix multiplication.

Hardware Characteristics

This design is:

✓ fully parallel

✓ pipelinable

✓ combinational

- ✓ low latency
- ✓ optimized for FPGA LUT mapping
- ✓ IoT-friendly low power

Operations used: XOR

shift

conditional **XOR**

No multipliers required.

All arithmetic is bit-level.

Security Role

MixColumns ensures:

inter-byte diffusion avalanche propagation

resistance to differential cryptanalysis

resistance to linear cryptanalysis

It spreads changes across the state.

One-bit input change affects many output bits.

Architectural Advantages

Direct GF arithmetic avoids:

✗ lookup tables

✗ RAM blocks

✗ large ROMs

This reduces: area power

memory

footprint

Ideal for: IoT AES cores embedded

systems

FPGA lightweight designs

ASIC implementations

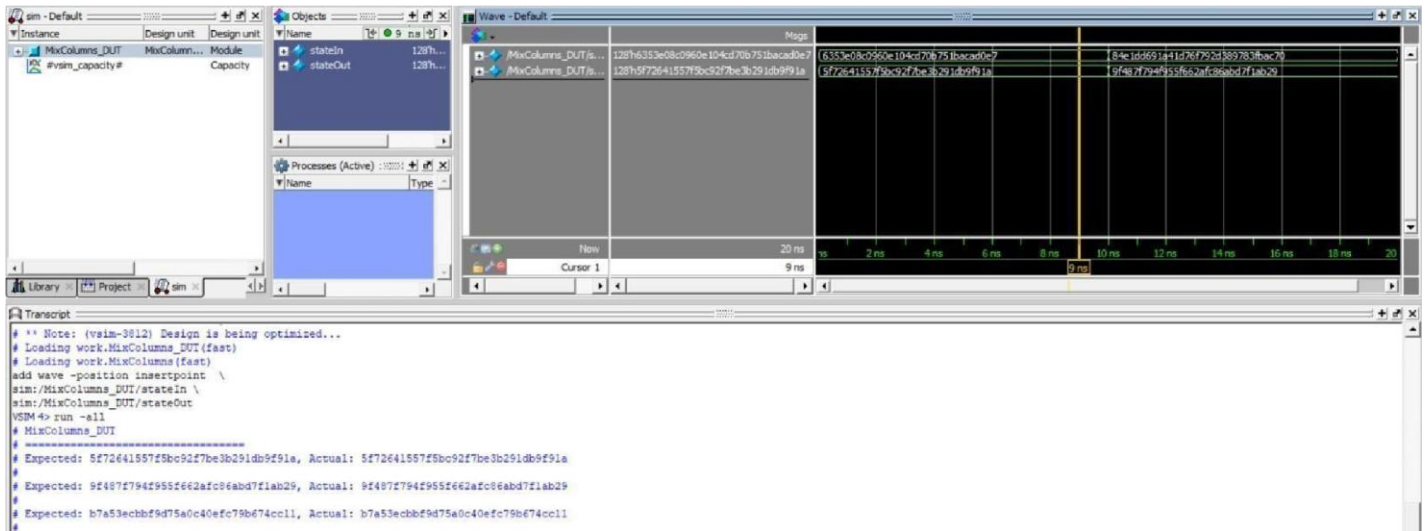


Figure 9 Test Bench MixColumns